Coinsult

Request your audit at coinsult.net

Advanced Manual
Smart Contract Audit

March 24, 2026

x.com/coinsultaudits

t.me/coinsult_tg

Audit requested by

Zylo Vortex | Milestone

0x0000000000000000000000000000000000000000

Global Overview

Manual Code Review

In this audit report we will highlight the following issues:

Vulnerability Level	Total	Pending	Acknowledged	Resolved
Informational	2	0	0	2
Low-Risk	4	1	0	3
Medium-Risk	8	1	1	6
Critical-Risk	5	0	0	5

Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

Vulnerability Level	Description
Informational	Does not compromise the functionality of the contract in any way.
Low-Risk	Won't cause any problems, but can be adjusted for improvement.
Medium-Risk	Will likely cause problems and it is recommended to adjust.
Critical-Risk	Will definitely cause problems, this needs to be adjusted.

Audit Summary

Project	Zylo Vortex | Milestone
Website	https://powerunit.live/
Blockchain	BSC (BEP-20)
Source Code	https://bscscan.com/address/0x0000000000000000000000000000000
Contract Address	0x0000000000000000000000000000000000000

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Audit Scope

Coinsult was commissioned to perform an audit based on the provided code. Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

Used tools

Slither: Solidity static analysis frameworkRemix: IDE Developer ToolCWE: Common Weakness EnumerationSWC: Smart Contract Weakness Classification and Test CasesDEX: Testnet Blockchains

Table of Contents

Global Overview 2

Audit Summary 3

Audit Scope 4

Detailed Findings

Unsafe ERC20 transfers (no return check) 6

DoS risk: unbounded loops over user locks 7

DoS risk: unbounded month processing loop 8

Bug: lastProcessedMonth not advanced correctly 9

claimLockedMilestonePagination can revert 10

Centralization: owner can change start time 11

Centralization: owner can drain any tokens 12

Centralization: owner controls milestone payouts 13

Centralization: owner can pick top winners payout 14

tx.origin restriction breaks smart wallets 15

External dependency trust: ZyloPowerUp 16

Unbounded milestone unlock loop (1..10 calls) 17

Partner uniqueness is global and permanent 18

Partner uniqueness check is ineffective 19

No validation of milestone index bounds 20

Hardcoded burn address is changeable variable 21

Owner can set MAX_MONTHS_PER_CALL too high 22

Typo: getCurrentMont naming 23

PeriodDeposit mapping in struct is opaque 24

Unsafe ERC20 transfers (no return check)

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-001: Resolved	Contract uses IERC20.transfer without checking the returned boolean. Non-standard tokens (or tokens that return false) can cause silent failures, breaking accounting and potentially locking user rewards/fees while state updates still occur.

Code Snippet

```
token.transfer(owner(), fee);
token.transfer(partners[i].account, share);
token.transfer(_user, remaining);
milestoneToken.transfer(_user, amount);
```

Recommendation

Use OpenZeppelin SafeERC20 (safeTransfer/safeTransferFrom) or require(token.transfer(...) == true). Apply to all token transfers including milestoneTopUser and emergencyWithdrawToken.

Note

All ERC20 transfers in this version use SafeERC20.safeTransfer (owner fee, partner shares, user remainder, burn, emergencyWithdrawToken). No raw IERC20.transfer usage found.

DoS risk: unbounded loops over user locks

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-002: Resolved	claimLockedMilestone and EliteAmount iterate over all PartnerLock entries. As locks grow, calls can exceed gas limits, preventing users from claiming vested tokens (funds effectively stuck). Attackers can also bloat their own locks via repeated actions.

Code Snippet

```
PartnerLock[] storage locks = partnerLockedMilestone[_user][_unit];
for (uint256 i = 0; i < locks.length; i++) { ... }
```

Recommendation

Add pagination (claim by index range), or track an active lock pointer and stop iterating over fully claimed locks. Consider merging locks per month/unit to cap array growth.

Note

User lock processing is bounded via pagination: EliteAmountPagination(from,to) and claimLockedMilestonePagination(from,to) iterate only over the provided range.

DoS risk: unbounded month processing loop

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-003: Resolved	_autoProcessPreviousMonth loops from lastProcessedMonth to currentMonth-1. If a user waits many months, claimX may run out of gas and become uncallable, blocking claims and partner distributions.

Code Snippet

```
for (uint256 m = lastMonth; m <= endMonth; m++) { ... }
```

Recommendation

Process at most N months per call and store progress, or provide a separate public function to process months in batches. Also fix lastProcessedMonth update logic (see AUD-004).

Note

_autoProcessPreviousMonth now caps work per call using MAX_MONTHS_PER_CALL and a processed counter in the month loop.

Bug: lastProcessedMonth not advanced correctly

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-004: Resolved	Inside the month loop, lastProcessedMonth is set to m (not m+1). Next call re-processes the same month (even if monthlyClaimX was zeroed), wasting gas and increasing DoS likelihood. It also never marks skipped months as processed when milestone not unlocked.

Code Snippet

```
lastProcessedMonth[_user][_unit] = m;
monthlyClaimX[_user][_unit][m] = 0;
```

Recommendation

Set lastProcessedMonth to m+1 after processing a month. For skipped months, still advance progress (or store a separate cursor) to avoid repeatedly scanning old months.

Note

lastProcessedMonth is advanced as m + 1 on every iteration (even when month is skipped), preventing re-processing/scanning the same month repeatedly.

claimLockedMilestonePagination can revert

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-016: Resolved	claimLockedMilestonePagination computes elapsed = block.timestamp - locks[i].startTime without checking startTime. If called in the same block as lock creation (or if startTime > now), subtraction underflows and reverts, blocking claims for ranges that include such locks.

Code Snippet

```
uint256 elapsed = block.timestamp - locks[i].startTime;
uint256 vested;
if (elapsed >= locks[i].duration) { ... }
```

Recommendation

Mirror the guard used in EliteAmountPagination: if (block.timestamp <= locks[i].startTime) continue; or clamp elapsed to 0. This prevents underflow and allows claiming other locks in the range.

Note

claimLockedMilestonePagination now guards underflow by skipping locks where block.timestamp <= locks[i].startTime before subtracting.

Centralization: owner can change start time

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-005: Resolved	Owner can arbitrarily change milestoneStartTime, which changes month calculations and can affect unlock/processing logic and user expectations. This can be used to accelerate/rewind periods and impact burns/locks and milestone eligibility.

Code Snippet

```
function setMilestoneStartTime(uint256 _time) public onlyOwner{
    milestoneStartTime = _time;
}
```

Recommendation

Remove this function after deployment, or restrict it (one-time set, timelock, only before launch). Emit an event and document governance controls.

Note

No owner setter exists for milestoneStartTime. It is initialized once from ZyloPowerUp.inceptStartTime() in the constructor.

Centralization: owner can drain any tokens

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-006: Pending	emergencyWithdrawToken lets the owner transfer any ERC20 from the contract to themselves. Users' expected rewards/locked funds can be removed, creating a rug-pull risk.

Code Snippet

```
function emergencyWithdrawToken(address _token, uint256 _amount) external onlyOwner {
    IERC20(_token).transfer(owner(), _amount);
}
```

Recommendation

Use a timelock/multisig, limit to non-core tokens, or only allow withdrawing excess balances above tracked liabilities. Emit events and publish policy.

Note

emergencyWithdrawToken remains onlyOwner and can withdraw any ERC20 except the main reward token. This still allows draining other tokens held by the contract (centralization/rug risk), albeit with a main-token restriction.

Centralization: owner controls milestone payouts

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-007: Resolved	Owner decides milestoneToken address and arbitrary reward amounts via giveMilestonePowerUp. This allows selective payouts, under/over-paying, or using a malicious token. Users cannot independently claim milestone rewards.

Code Snippet

```
function setmilestoneToken(address _token) external onlyOwner { ... }
function giveMilestonePowerUp(address _user,uint256 index,uint256 amount) external onlyOwner
{ ... }
```

Recommendation

Make rewards deterministic and claimable by users, or enforce capped amounts per milestone. Use multisig/timelock for admin actions and emit detailed events.

Note

Admin-driven milestone payout functions (setMilestoneToken/giveMilestonePowerUp) are not present in this codebase, removing the discretionary milestone payout control described in the finding.

Centralization: owner can pick top winners payout

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-008: Resolved	milestoneTopUser is onlyOwner and transfers any token to the computed winner. Owner chooses token and amount and can call at any time, enabling unfair distribution or draining contract balances.

Code Snippet

```
function milestoneTopUser(..., address winnertoken, uint256 amount) external onlyOwner {
    IERC20(winnertoken).transfer(winner, amount);
}
```

Recommendation

Hardcode allowed reward tokens and amounts per period, or require on-chain funding and deterministic distribution. Use timelock/multisig and add checks that contract has sufficient balance.

Note

milestoneTopUser (owner-chosen token/amount payout) is not present in this version, so the discretionary top-winner payout vector no longer applies.

tx.origin restriction breaks smart wallets

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-009: Resolved	Using require(msg.sender == tx.origin) blocks contract-based wallets (Gnosis Safe, account abstraction) and some relayers, reducing usability. It is not a reliable security control and can cause users to be unable to claim.

Code Snippet

```
require(msg.sender == tx.origin, "External Access Not Allowed");
```

Recommendation

Remove tx.origin checks. If you need bot protection, use signatures (EIP-712), allowlisted relayers, rate limits, or other anti-abuse mechanisms.

Note

No tx.origin checks exist; functions rely on msg.sender and standard access control.

External dependency trust: ZyloPowerUp

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-010: Acknowledged	Critical logic depends on external ZyloPowerUp/ZyloPowerUpDrop values. If those contracts are upgradeable/compromised, they can manipulate unlocks, claim amounts, and top-user tracking, affecting payouts and burns/locks.

Code Snippet

```
ZyloPowerUp.totalSelfClaimInUnit(...)
ZyloPowerUp.getMilestoneCount(...)
ZyloPowerUpDrop.totalSelfClaimInUnit(...)
```

Recommendation

Document trust assumptions. If possible, reference immutable, non-upgradeable contracts or add governance/timelock for changing addresses. Consider sanity checks and pause mechanisms.

Note

> Core accounting/unlock logic still depends on external ZyloPowerUp/ZyloPowerUpDrop. Addresses are immutable, but if those contracts are compromised/upgradeable, they can still manipulate totals and milestone unlock behavior.

Unbounded milestone unlock loop (1..10 calls)

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-017: Resolved	updateMilestoneReq loops index=1..10 and makes many external calls to ZyloPowerUp.getPowerUpLength/getMilestoneCount. If ZyloPowerUp is expensive or reverts intermittently, it can DoS milestone updates. Also, only ZyloPowerUp can call it, so users cannot retry independently.

Code Snippet

```
for (uint256 index = 1; index <= 10; index++) {
  if (ZyloPowerUp.getPowerUpLength(_user, 0) < s.sparkup) continue;
  ...
}
```

Recommendation

Consider batching (process a subset of indices per call) or caching counts from ZyloPowerUp in a single call. Ensure ZyloPowerUp cannot grief by reverting; optionally add try/catch with safe defaults.

Note

updateMilestoneReq now caches ZyloPowerUp.getPowerUpLength/getMilestoneCount for units 0..4 once each and uses try/catch with safe defaults, reducing repeated external calls and mitigating revert-based DoS.

Partner uniqueness is global and permanent

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-018: Resolved	isPartnerUsed is global across all users and never cleared. A partner address can only ever be used once system-wide, and there is no removal/update path. This can permanently lock users out of adding intended partners and can be griefed by front-running partner registration.

Code Snippet

```
require(!isPartnerUsed[_accounts[i]], "Partner already used");
...
isPartnerUsed[_accounts[i]] = true;
```

Recommendation

If uniqueness should be per-user, key by (user, partner). If global uniqueness is intended, add a controlled removal/update mechanism (with events) or a partner opt-in signature to prevent griefing.

Note

Partner uniqueness is no longer global/permanent; it is enforced per user (isPartnerUsedPerUser[user][partner]) which avoids system-wide lockout/griefing described previously.

Partner uniqueness check is ineffective

Low-Risk *- Won't cause any problems, but can be adjusted for improvement.*

Error Code	Description
AUD-011: Resolved	isPartnerUsed is checked but never set (line commented). This makes the 'Partner already used' protection ineffective, allowing the same partner address to be reused across many users, contrary to apparent intent.

Code Snippet

```
require(!isPartnerUsed[_accounts[i]], "Partner already used");
// isPartnerUsed[_accounts[i]] = true;
```

Recommendation

Either remove isPartnerUsed logic entirely (if reuse is allowed) or set it when adding partners. Also consider allowing partners to be updated/removed with clear rules.

Note

Partner uniqueness is now enforced per-user via isPartnerUsedPerUser[_user][_accounts[i]] and is set to true when adding partners.

No validation of milestone index bounds

Low-Risk *- Won't cause any problems, but can be adjusted for improvement.*

Error Code	Description
AUD-012: Resolved	Functions accept arbitrary milestone index values. While mappings prevent out-of-bounds storage errors, it can create inconsistent state (e.g., unlocking/claiming for unexpected indices) and complicate off-chain assumptions.

Code Snippet

```
function giveMilestonePowerUp(address _user, uint256 index, uint256 amount) ...
```

Recommendation

Add require(index>=1 && index<=10) where applicable (updateSelfReq/updateTeamMilestoneReq/giveMilestonePowerUp/updateMilestoneReq) to enforce intended milestone set.

Note

The previously cited index-bound issue related to admin milestone payout/update functions; those functions are absent here. updateMilestoneReq itself iterates index 1..10 only.

Hardcoded burn address is changeable variable

Low-Risk *- Won't cause any problems, but can be adjusted for improvement.*

Error Code	Description
AUD-013: Resolved	BURN_ADDRESS is a public variable (not constant/immutable). While no setter exists now, future upgrades or inherited contracts could change it. Users may assume burns are permanent but could be redirected.

Code Snippet

```
address public BURN_ADDRESS = address(0x3b0a...);
```

Recommendation

Make burn address immutable/constant if it must never change. If it should be configurable, add a setter with timelock/multisig and emit an event.

Note

BURN_ADDRESS is immutable and set once in the constructor.

Owner can set MAX_MONTHS_PER_CALL too high

Low-Risk *- Won't cause any problems, but can be adjusted for improvement.*

Error Code	Description
AUD-019: Pending	Owner can increase MAX_MONTHS_PER_CALL arbitrarily. Setting it too high can reintroduce gas-heavy processing in claimX via _autoProcessPreviousMonth, increasing revert risk and harming UX.

Code Snippet

```
function setMaxMonthsPerCall(uint256 _newMax) external onlyOwner {
  MAX_MONTHS_PER_CALL = _newMax;
}
```

Recommendation

Cap MAX_MONTHS_PER_CALL to a reasonable maximum (e.g., 12) or make it immutable after deployment. Alternatively, enforce a gas-aware upper bound.

Note

MAX_MONTHS_PER_CALL remains a mutable public variable (default 3) with no hard cap shown in this code. Even though no setter is present here, mutability still allows future changes (e.g., via upgrade/derived contract) and reintroducing gas-heavy processing.

Typo: getCurrentMont naming

Informational *- Does not compromise the functionality of the contract in any way.*

Error Code	Description
AUD-014: Resolved	Function name getCurrentMont appears to be a typo. This is not a security issue but can confuse integrators and auditors and lead to incorrect usage in external tooling.

Code Snippet

```
function getCurrentMont() public view returns(uint256 month){ ... }
```

Recommendation

Rename to getCurrentMonth for clarity (or add an alias) and update any dependent contracts/interfaces accordingly.

Note

Function is named getCurrentMonth() in this version.

PeriodDeposit mapping in struct is opaque

Informational *- Does not compromise the functionality of the contract in any way.*

Error Code	Description
AUD-015: Resolved	PeriodDeposit contains a mapping, which cannot be returned directly and is hard to inspect on-chain. This reduces transparency for users verifying deposits/top users and can complicate monitoring.

Code Snippet

```
struct PeriodDeposit {
  address topUser;
  uint256 topAmount;
  mapping(address => uint256) userDeposits;
  bool milestoneDistributed;
}
```

Recommendation

Add view functions to read userDeposits for a period and to list/verify top user logic. Emit events on updates for easier off-chain indexing.

Note

View functions were added to read PeriodDeposit.userDeposits and top user/amount per period (monthly/weekly/daily and incept variants), improving transparency.

Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.